

Efficient Lookup-Table-Based Joint Noise Sampling for Differential Privacy

Sebastian David Albu

June 24, 2026

Motivation and Background

Secure Joint Computation & Privacy

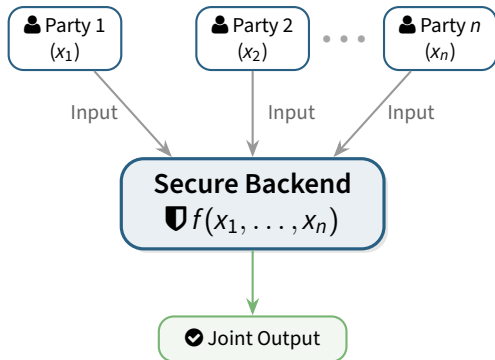
The Premise: Secure Joint Computation

The Goal

Multiple parties want to jointly compute an aggregate function $f(x_1, \dots, x_n)$ on their private data without revealing their local inputs to each other.

The "Secure Backend":

- We assume a generic secure computation engine.
- Examples: Secure Multi-Party Computation (MPC).
- **Constraint:** The backend supports basic linear math, but complex operations are highly constrained by bandwidth or memory.

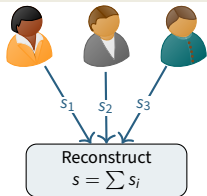


Secret Sharing: Additive Arithmetic vs. Binary Shares

Secret sharing splits a secret s into shares $\{s_i\}$ so that no subset below the threshold learns s , but authorized subsets can reconstruct $[EKR18]$.

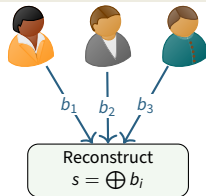
Arithmetic Shares $\langle r \rangle \pmod{p}$

$$s = \sum_{i=1}^n s_i \pmod{p}$$



Binary Shares $[r]$ (XOR)

$$s = \bigoplus_{i=1}^n s_i$$



The Goal: Mix math (+) and logic (<).

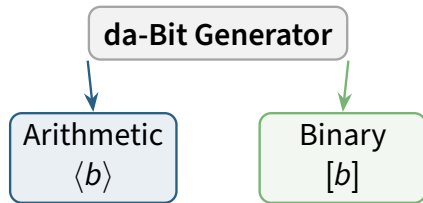
Doubly-Authenticated Bits [RW19]

A random bit $b \in \{0, 1\}$ shared
simultaneously in both domains:

$$(\langle b \rangle, [b])$$

Benefits:

- ✓ Fast Bit-Decomposition.
- ✓ Cheap domain switching.



The Catch

da-Bits are complex to generate [RW19].
MP-SPDZ [Kel20] lacks support for them in
some protocols.

→ Without them, we are restricted to
"Arithmetic Only" execution.

Differential Privacy

What is Differential Privacy (DP)?

The Gold Standard for Output Privacy

A mathematical guarantee that protects individuals in a dataset by ensuring the output does not leak their participation [Dwo+06].

The Guarantee (ϵ -DP):

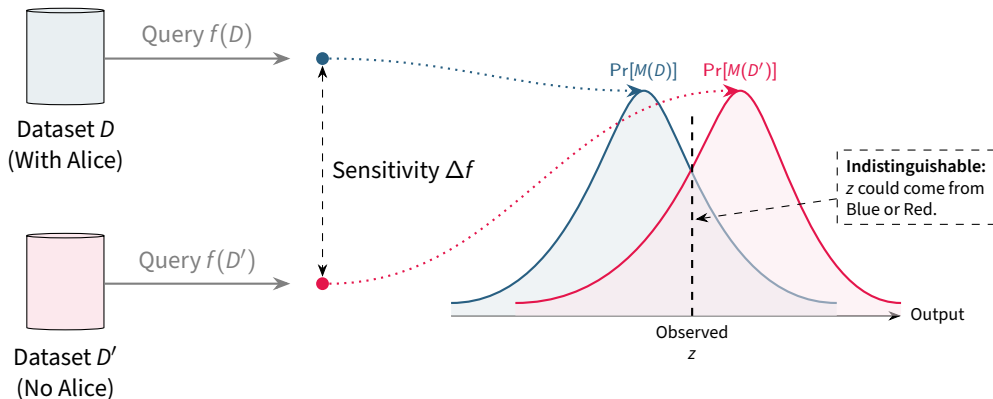
- **Plausible Deniability:** The result is effectively the same, regardless of your participation [MRS24].

How it works

We add mathematically calibrated **Noise** (Z) to the true function result.

$$\text{Output} = f(D) + Z$$

Visualizing the Privacy Guarantee



Real-World Scenario: Private AI Training

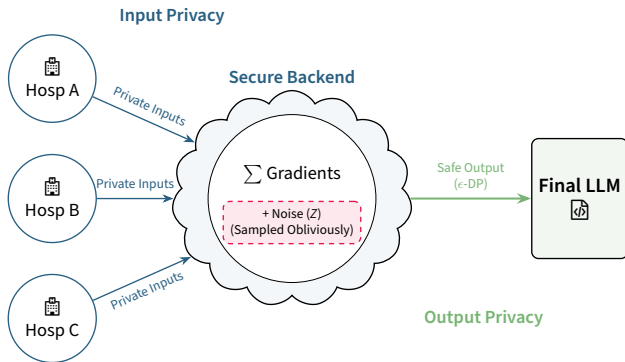
Scenario: 3 Hospitals training a shared LLM.

The Risks:

- 1 Input Leakage:** No hospital can see another's raw records. (Solved by e.g. Secure MPC)
- 2 Inversion:** Model must not "memorize" specific patients. (Solved by DP)

The Optimal Pipeline

Train securely, and inject DP Noise (Z)
internally before revealing the final model.



The Core Bottleneck

Generating DP noise (Laplace or Gaussian) is easy in plaintext, but incredibly difficult inside a secure backend.

Standard Algorithms Require [MRS25]:

- **Inverse Transform:** Requires $\ln(x)$ and floating point divisions.
- **Rejection Sampling:** Requires many unknown loops and active comparisons.
- **Bitwise Sampling:** Requires long sequences of non-linear state updates [ASR25; Eri+22; MRS24].

The Cost in Secure MPC:

- Linear operations (Addition) are effectively **free** [KS22].
- Non-linear operations (Multiplications, Comparisons) are **highly expensive**, bottlenecking network bandwidth and CPU caches [KS22; MRS25].
- ✗ Existing samplers require **Megabytes** of communication per sample [MRS25]!

The Solution: Lookup-Tables & Dot-Products [MRS25]

What is a Lookup-Table (LUT)?

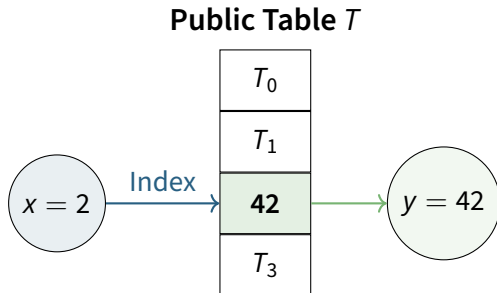
Definition

A pre-computed array where:

- **Input:** An index x .
- **Output:** The value $T[x]$.

Why use them in Secure Computation?

- The backend is fast at linear math $(+, -)$.
- The backend is **slow** at non-linear math.
- **Solution:** Pre-calculate the hard math into a public table T and just "look up" the result [MRS25].



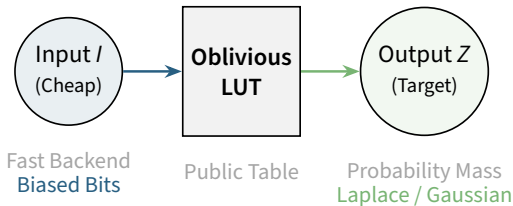
Core Concept: The LUT as a Probability Mapper

The Logic:

- **Input:** Sample a "cheap" index I (fast).
- **Translation:** Use a public LUT to map $I \rightarrow Z$.
- **Output:** The complex target noise Z .

Instance Generation

The LUT is inherently **distribution-agnostic**. We pre-fill it offline using a greedy algorithm to perfectly match the target probability mass, ensuring cryptographic accuracy ($SD \approx 2^{-40} (\lambda = 40)$) [MRS25].



How do we ensure the output distribution is correct?

1. Index Sampling (Interactive)

We use **Skewed Bernoulli Bits** [MRS25].

- $I = b_1 || b_2 || \dots || b_k$
- $b_i \sim \text{Ber}(2^{-c})$ (Biased).
- **Why?** Concentrates indices near 0, matching the "shape" of Geometric/Gaussian distributions.

2. Table Filling (Instance Generation)

We use a **Greedy Algorithm** $\mathcal{A}_{\text{Fill}}$ [MRS25].

- Sort indices by probability.
- Sort target values by probability.
- **Map** the most likely index to the most likely output.
- Fills gaps via exhaustive search.

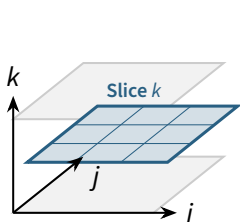
Optimization: Dimensionality Reduction ($d = 1 \rightarrow 3$)

A flat LUT of size $N = 2^{24}$ is too slow to scan linearly. We drastically reduce communication overhead by increasing the tensor dimension d [MRS25].

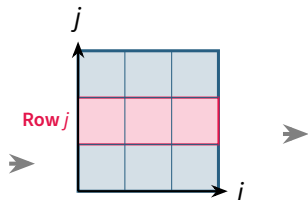
1D Vector ($d = 1$):
Size: 16.7 million elements.
⚠ **Cost: ≈ 2 MB / sample.**

2D Matrix ($d = 2$):
 4096×4096 grid.
↔ **Cost: ≈ 1 KB / sample.**

3D Cube ($d = 3$):
 $256 \times 256 \times 256$ tensor.
🚀 **Cost: Just 96 Bytes!**



(a) 3D Tensor



(b) 2D Matrix



(c) 1D Vector



(d) Scalar

MAESTRO [Mor+24] optimizes LUTs by decoupling the workload.

1. Offline Phase (Heavy)

- Pre-generates correlated randomness.
- Creates a random **One-Hot Vector** $\langle e(r) \rangle$.
- (A vector with all zeroes and just a one at index r)

2. Online Phase (Light)

- **Goal:** Linearize the lookup.
- Mask the secret index v with random r :

$$c = v \oplus r$$

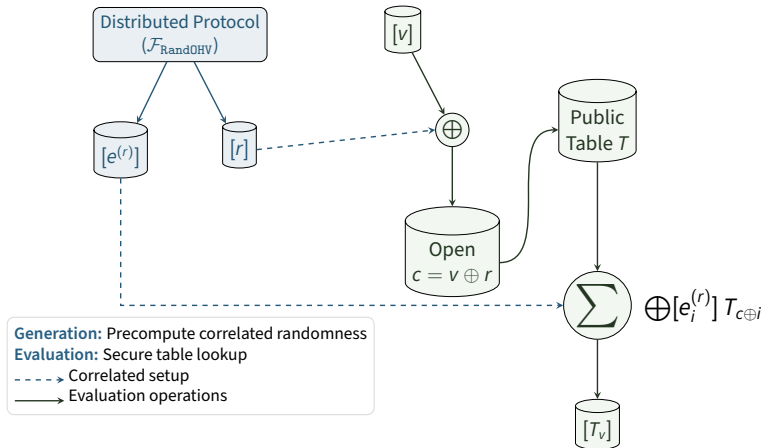
- Single round of communication.

Result: Sublinear online communication $\mathcal{O}(\log n)$

LUTs in MAESTRO: The Strategy

• Randomness Generation

• Table Evaluation



The Frameworks: RSS & SSS

We deploy on two secret sharing schemes:

- **RSS (Rep3)** [MR18; ISN89]: 3-party. Highly optimized, natively supports mixed-domain (da-Bits [RW19; Kel20]).
- **Shamir (SSS)** [Sha79]: N -party. Highly scalable, but restricted to pure Arithmetic ($\mathbb{F}_p$).

The Mathematical Superpower

Both schemes share: **Efficient Dot Products.**

- **Standard MPC:** $\vec{A} \cdot \vec{B}$ of length N takes N interactive multiplications.
- **RSS/SSS:** Parties compute partial cross-terms locally, communicating once to reshare.
- ✓ **Cost:** $\mathcal{O}(1)$ interaction!

Impact on LUTs

Our 3D-Cube lookup evaluates large tensors via dot-products. Thanks to RSS/SSS, interactive communication **does not grow linearly** with table size! [MRS25]

Binomial Indexing and the "Domain Sandwich"

The Problem: Flat Bernoulli Indices

Bernoulli sampling generates a structurally flat index.

- ↗ To map this flat index to a long-tailed Laplace curve, the LUT must explicitly expand.
- 🗄 This forces large table sizes ($K \geq 24$) at strict privacy regimes.

The Pivot: Binomial Sums

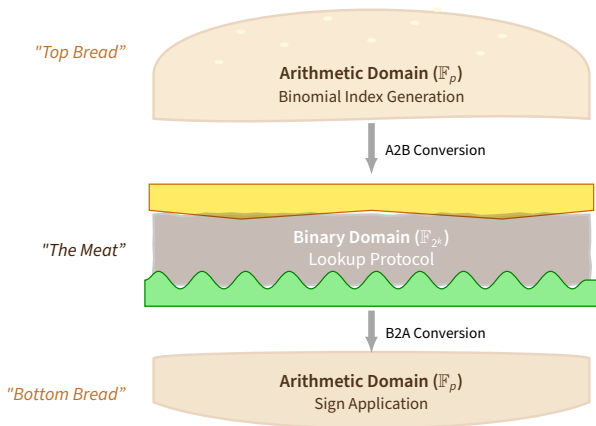
$\mathcal{B}(N, 0.5)$ is the sum of N uniform coins.

- ⇒ **Trade-off:** Upfront arithmetic cost to generate $\mathcal{F}_{\text{Bit}}$.
- ↗ **The Advantage:** The native bell-shape naturally compresses the required 3D-LUT size.

The "Domain Sandwich" Architecture

The Strategy: Use each domain's native strength.

- ⊕ **Top Bun (Arithmetic):**
Index generation ($\sum$ coins) is free.
- ⚠ **The Bottleneck (Π_{A2B}):**
Converting to Binary is highly interactive!
- ⚙ **The Meat (Binary):**
Fast bitwise LUT lookups.
- ⇒ **Bottom Bun (Arithmetic):**
Convert back (Π_{B2A}) to easily apply a random ± 1 sign.



The Interactive Bottleneck

Π_{A2B} conversions dominate interactive latency, especially for small sample batches.



Goal:

Shift the domain crossing entirely into the pre-computation phase!

The Optimization (Pre-Computation):

- Pre-generate a mixed-domain tuple via da-Bits: $(\langle r \rangle, [e^{(r)}])$
- *An arithmetic offset, and its exact binary One-Hot Vector equivalent.*

The Interactive Evaluation:



Mask & Open:

Hide the index: $d = \text{Open}(\langle l \rangle + \langle r \rangle)$.



Local Shift:

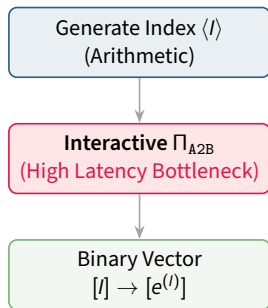
Publicly circular shift the pre-generated vector by d .



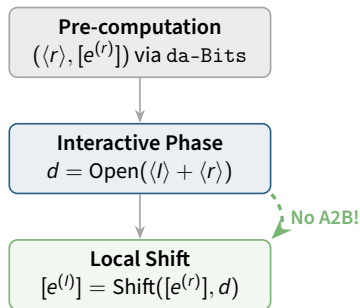
Impact: Zero interactive Π_{A2B} conversions!

Visualizing the Optimization

The "Old" Way



Optimized (New)



Implementation Results

The Pure Arithmetic Memory Exhaustion

We attempted to run the 3D-Cube collapse entirely in the Arithmetic Domain ($\mathbb{F}_p$) to support universal backend protocols.

At $K = 24$ (16.7 million elements), the 64-bit verification overhead caused **CPU cache exhaustion and triggered an OOM (Out-of-Memory)**.

The Domain Sandwich Prevails

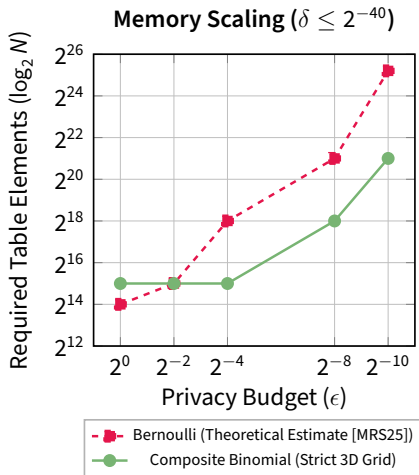
By shifting the tensor collapse into the Binary Domain ($\mathbb{F}_{2^k}$), we utilize native **bit-packing**.

This effortlessly absorbs the geometric penalty of the 65, 536 interactive cross-term multiplications required for a 3D tensor collapse.

Avoiding Exponential Growth

To maintain a strict DP security bound ($\delta \leq 2^{-40}$), the lookup table must capture the extreme tails of the distribution.

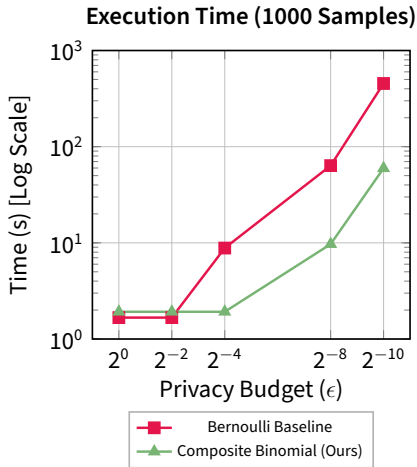
- **Bernoulli Baseline:** Requires explicitly wider tables, scaling beyond 2^{24} elements for tight privacy [MRS25].
- **Composite Binomial:** By naturally mimicking the bell curve, it maps the required precision into drastically smaller domains.



The Scaling Problem

As privacy constraints tighten ($\epsilon \rightarrow 2^{-10}$), the distribution's tail grows longer.

- **Bernoulli:** Explicitly mapping these tiny fractional probabilities forces the table size (K) to expand exponentially.
- **Binomial Composite:** The native bell-shape inherently mimics the target curve, requiring drastically smaller table dimensions.



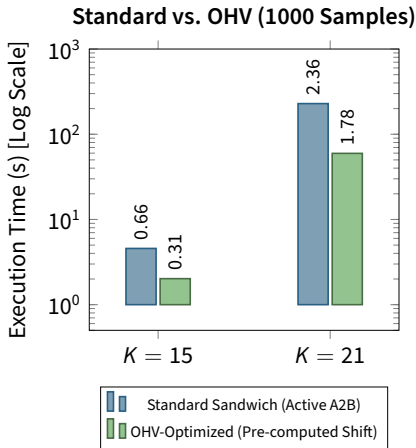
Removing the Bottleneck

The Standard Sandwich requires an active Π_{A2B} protocol to convert the arithmetic index into binary shares.

- **The Penalty:** Active Π_{A2B} conversions require interactive secure binary subtraction circuits.

The OHV Optimization

By consuming pre-computed mixed-domain tuples, we replace the interactive A2B circuit with a single local circular shift.



Conclusion and Future Work

Conclusion: Summary of Contributions

1. Framework Integration

Bridged the gap between theory and practice by integrating offline-heavy LUT probability mapping into MP-SPDZ [Kel20].

2. The Binomial Sandwich

Replaced flat fractional sampling with **Composite Binomial Indexing**, naturally matching the Laplace curve to prevent memory explosion.

3. The OHV Optimization

Eliminated the interactive Π_{A2B} bottleneck by leveraging pre-computed mixed-domain tuples, replacing heavy conversions with a single local shift.

1. Geometric Indexing

- **Concept:** Use a Geometric base distribution instead of Binomial.
- **Goal:** The exponential decay naturally matches the Laplace tail, potentially compressing the LUT size (K) even further for extreme privacy budgets ($\epsilon \ll 2^{-10}$).

2. Secure Rejection Sampling

- **Concept:** Detect when an index falls outside table bounds and securely re-sample.

3. Dishonest-Majority Adaptation

- **Current Limitation:** Our incredible dot-product efficiency relies on **Honest-Majority** schemes.
- **The Challenge:** Extending this architecture to **Dishonest-Majority** models (e.g., SPDZ_{2^k}).
- **The Goal:** Adapting the $\mathcal{O}(N^{1/3})$ tensor collapse to operate efficiently under strict MAC authentications without triggering CPU/RAM exhaustion.

Efficient Lookup-Table-Based Joint Noise Sampling for Differential Privacy

Sebastian David Albu

June 24, 2026

Appendix A: Performance Baseline Master's Project

Benchmark: Generating 1000 Discrete Laplace samples at high privacy ($\epsilon = 0.1$).

Metric	Dynamic Baseline (FDL1)	Our Work (Composite 3D-OHV)
Execution Time	221 Seconds	2.02 Seconds ($\approx 100\times$ faster)
Network Data	> 14 Gigabytes	23.9 Megabytes ($\approx 600\times$ smaller)
Table Size (K)	N/A (Runtime Evaluation)	$K = 15$ (Highly Compressed)
Trust Model	Dishonest-Majority	Honest-Majority (Malicious)

- **The 2^{15} Compression:** By using the bell-shaped Composite Binomial index, we successfully compressed the tensor dimensions without losing DP guarantees ($\delta \leq 2^{-40}$).
- **Conclusion:** Pre-computed, discrete-space combinatorial sampling is a highly scalable foundation for high-throughput DP.

Appendix B: Toy Example of the OHV Shift

Goal: Securely extract the value at secret index I from a public table T .

Suppose $T = [T_0, T_1, T_2, T_3]$. Our secret index is $I = 1$ (we want to output T_1).

1. Offline Setup (The Secret)

The backend pre-generates a random mask r and its secret-shared One-Hot Vector $[e^{(r)}]$.

Let mask $r = 2$.

$$[e^{(2)}] = [0, 0, \mathbf{1}, 0]$$

2. Online Masking (The Public Info)

We hide I with r using bitwise XOR ($\oplus$) and open the result to all parties.

$$c = I \oplus r \implies 1 \oplus 2 = \mathbf{3}.$$

3. The Public Shift

Every party locally reorders the public table T using the public mask $c = \mathbf{3}$.

$$T_{\text{shift}} = [T_{\mathbf{3} \oplus 0}, T_{\mathbf{3} \oplus 1}, T_{\mathbf{3} \oplus 2}, T_{\mathbf{3} \oplus 3}]$$

$$T_{\text{shift}} = [T_3, T_2, \mathbf{T_1}, T_0]$$

4. The Secret Dot Product

Multiply the secret OHV by the shifted table.

$$\begin{aligned} [e^{(2)}] \cdot T_{\text{shift}} &= [0, 0, \mathbf{1}, 0] \cdot [T_3, T_2, \mathbf{T_1}, T_0] \\ &= 0 + 0 + \mathbf{T_1} + 0 = \mathbf{T_1} \end{aligned}$$

Result: We extracted T_1 , but $c = 3$ looks random, keeping $I = 1$ secure!

Appendix C: Topology Scaling (1D vs. 2D vs. 3D)

Table Size K	Batch Samples	1D Vector		2D Matrix		3D Cube	
		MB	Time (s)	MB	Time (s)	MB	Time (s)
$K = 15$	1000	268.63	18.21	<i>N/A (Odd K)</i>	<i>N/A</i>	15.61	1.83
$K = 18$	1000	<i>OOM</i>	–	18.82	8.38	41.26	8.83
$K = 21$	1000	<i>OOM</i>	–	<i>N/A (Odd K)</i>	<i>N/A</i>	141.66	63.54
$K = 24$	1000	<i>OOM</i>	–	104.93	1428.78	539.02	454.12

- **The 1D Memory Cliff:** Fails rapidly due to massive contiguous RAM requirements for verification buffers.
- **The 2D Cache Exhaustion:** While 2D handles $K = 18$ well, at $K = 24$ the dot-product vectors (length 4096) exceed CPU cache, causing a massive 24-minute latency. 3D compresses the dot-product to length 256.

Appendix D: Network Scaling ($n \geq 3$ Parties)

Table Size K	Batch Samples	3 Parties		4 Parties		5 Parties		6 Parties	
		MB	Time (s)	MB	Time (s)	MB	Time (s)	MB	Time (s)
$K = 18$	500	78.77	26.58	155.95	28.88	232.19	31.56	308.41	34.47
	1000	156.89	53.24	310.80	58.20	463.26	63.52	614.91	68.42
$K = 21$	500	280.37	195.26	558.88	218.12	836.98	236.22	1114.29	258.91
	1000	560.08	392.91	1116.68	436.33	1672.46	541.04	2226.66	507.21

- **Predictable Time Degradation:** Adding parties increases the polynomial degree for Shamir Secret Sharing, but time scales gracefully ($\approx 28\%$ penalty from 3 to 6 parties).
- **Multiplicative Data Growth:** Because shares must be transmitted to a larger subset of peers, the network volume scales multiplicatively.

Appendix E: Continuous vs. Discrete Scaling

Privacy ϵ	Continuous Base (FDL1)		Standard Sandwich		OHV-Optimized (Ours)	
	Time (s)	Data (MB)	Time (s)	Data (MB)	Time (s)	Data (MB)
$\epsilon = 1.0$	116.15	4,635.40	4.57	30.12	2.02	23.92
$\epsilon = 0.25$	148.52	6,747.14	4.57	30.12	2.02	23.92
$\epsilon = 0.1$	221.14	14,033.70	4.57	30.12	2.02	23.92

- **The FDL1 Vulnerability:** To maintain a 2^{-40} security margin against tail-end truncation as $\epsilon \rightarrow 0$, continuous samplers must unroll increasingly deep geometric coin-flipping circuits, causing exponential data growth [Eri+22; ASR25].
- **The LUT Stability:** By moving the mathematical precision to the offline instance generation, our 3D tensor ($K = 15$) provides a constant, flat evaluation cost regardless of the ϵ privacy constraint [MRS25].

Appendix F: Hybrid vs. Pure Arithmetic Domains

Batch Size at $K = 24$	Hybrid 3D Cube (Domain Sandwich)		Pure Arithmetic 3D Cube	
	Time (s)	Data (MB)	Time (s)	Data (MB)
100 Samples	44.08	59.38	462.92	216.14
300 Samples	133.39	165.97	1366.12	648.10
500 Samples	226.89	272.55	2262.51	1080.11
1000 Samples	454.12	539.02	OOM (Crash)	–

- **The Data Amplification Penalty:** Evaluating purely in the prime field requires a full 64-bit element for every share. The Hybrid binary domain packs 64 operations into a single register (SIMD bit-packing).
- **The Memory Wall:** The $\mathcal{O}(N^{2/3})$ geometric tensor collapse requires 65,536 interactive multiplications per sample. Without bit-packing, buffering the verification triples for this math exhausts system RAM (OOM) at 1000 samples.

Architecture (1000 Samples, $K = 21$)	Execution Time (s)		Network Data (MB)	
	Semi-Honest	Malicious	Semi-Honest	Malicious
Standard Sandwich	56.06	59.66	148.81	165.37
OHV-Optimized Sandwich	59.94	60.36	148.36	159.08

- **Negligible Time Penalty:** The wall-clock penalty for malicious security is incredibly low ($\approx 0.7\%$ for OHV). Cryptographic $\mathcal{F}_{\text{Verify}}$ checks are efficiently pipelined in the background while the CPU computes the heavy tensor collapse.
- **Minimizing Data Overhead:** The Standard Sandwich requires an extra 16.5 MB of network bandwidth for active verification during the Π_{A2B} conversion. The OHV approach bypasses this, successfully cutting the malicious data penalty in half.

- [ASR25] Sebastian David Albu, Fabian Schmid, and Christian Rechberger. **D-LAMP: Discrete Laplacian MPC Noise-Generating Protocols for Differentially Private Computation: Implementing and Comparing Novel Noise Generation Protocols for the Discrete Laplacian Distribution for Differentially Private Secure Multiparty Computation.** Internal publication. Master's Project. Graz, Austria: Graz University of Technology, Institute of Information Security (ISEC), 2025.
- [Dwo+06] Cynthia Dwork et al. **Our Data, Ourselves: Privacy Via Distributed Noise Generation.** Advances in Cryptology - EUROCRYPT 2006, 25th Annual International Conference on the Theory and Applications of Cryptographic Techniques, St. Petersburg, Russia, May 28 - June 1, 2006, Proceedings. Ed. by Serge Vaudenay. Vol. 4004. Lecture Notes in Computer Science. Springer, 2006, pp. 486–503. DOI: [10.1007/11761679%5C_29](https://doi.org/10.1007/11761679%5C_29). URL: https://doi.org/10.1007/11761679%5C_29.
- [EKR18] David Evans, Vladimir Kolesnikov, and Mike Rosulek. **A Pragmatic Introduction to Secure Multi-Party Computation.** *Found. Trends Priv. Secur.* 2.2-3 (2018), pp. 70–246. DOI: [10.1561/33000000019](https://doi.org/10.1561/33000000019). URL: <https://doi.org/10.1561/33000000019>.

- [Eri+22] Reo Eriguchi et al. **Efficient Noise Generation Protocols for Differentially Private Multiparty Computation.** *IACR Cryptol. ePrint Arch.* (2022), p. 1391. URL: <https://eprint.iacr.org/2022/1391>.
- [ISN89] Mitsuru Ito, Akira Saito, and Takao Nishizeki. **Secret sharing scheme realizing general access structure.** *Electronics and Communications in Japan (Part III: Fundamental Electronic Science)* 72.9 (1989), pp. 56–64.
- [Kel20] Marcel Keller. **MP-SPDZ: A Versatile Framework for Multi-Party Computation.** Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. 2020. DOI: [10.1145/3372297.3417872](https://doi.org/10.1145/3372297.3417872). URL: <https://doi.org/10.1145/3372297.3417872>.
- [KS22] Marcel Keller and Ke Sun. **Secure Quantized Training for Deep Learning.** International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA. Ed. by Kamalika Chaudhuri et al. Vol. 162. Proceedings of Machine Learning Research. PMLR, 2022, pp. 10912–10938. URL: <https://proceedings.mlr.press/v162/keller22a.html>.

- [Mor+24] Hiraku Morita et al. **MAESTRO: Multi-party AES using Lookup Tables.** *IACR Cryptol. ePrint Arch.* (2024). Accessed: 2025-05-22, p. 1317. URL: <https://eprint.iacr.org/2024/1317>.
- [MR18] Payman Mohassel and Peter Rindal. **ABY³: A Mixed Protocol Framework for Machine Learning.** Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018. Ed. by David Lie et al. ACM, 2018, pp. 35–52. DOI: [10.1145/3243734.3243760](https://doi.org/10.1145/3243734.3243760). URL: <https://doi.org/10.1145/3243734.3243760>.
- [MRS24] Fredrik Meisingseth, Christian Rechberger, and Fabian Schmid. **Practical Two-party Computational Differential Privacy with Active Security.** Cryptology ePrint Archive, Paper 2024/004. Accessed: 2025-02-28. 2024. URL: <https://eprint.iacr.org/2024/004>.
- [MRS25] Fredrik Meisingseth, Christian Rechberger, and Fabian Schmid. **Accelerating Multiparty Noise Generation Using Lookups.** Cryptology ePrint Archive, Paper 2025/805. Accessed: 2025-10-07. 2025. URL: <https://eprint.iacr.org/2025/805>.

- [RW19] Dragos Rotaru and Tim Wood. **MARbled Circuits: Mixing Arithmetic and Boolean Circuits with Active Security**. Progress in Cryptology - INDOCRYPT 2019 - 20th International Conference on Cryptology in India, Hyderabad, India, December 15-18, 2019, Proceedings. Ed. by Feng Hao, Sushmita Ruj, and Sourav Sen Gupta. Vol. 11898. Lecture Notes in Computer Science. Springer, 2019, pp. 227–249. DOI: [10.1007/978-3-030-35423-7_12](https://doi.org/10.1007/978-3-030-35423-7_12). URL: https://doi.org/10.1007/978-3-030-35423-7_12.
- [Sha79] Adi Shamir. **How to Share a Secret**. **Commun. ACM** 22.11 (1979), pp. 612–613. DOI: [10.1145/359168.359176](https://doi.org/10.1145/359168.359176). URL: <https://doi.org/10.1145/359168.359176>.

Efficient Lookup-Table-Based Joint Noise Sampling for Differential Privacy

Sebastian David Albu

June 24, 2026